📊 Results Summary - VLMs Disaster Classification

Last Updated: 2026-03-28 **Hardware:** MacBook Pro M4 Pro (24GB unified memory) **Backend:** PyTorch 2.11.0 + MPS (Metal Performance Shaders)

🎯 Executive Summary (TL;DR)

What we tested: Can Vision-Language Models (VLMs) automatically classify disaster images from social media?

Result: ✅ **YES - 82.65% accuracy** on 6-class damage classification

Best model: FuseLIP-MLP (image-only)

- Detects **human damage** with 94% precision (people injured/affected)
- Detects **floods** with 80% accuracy
- Struggles with **fires** (55% accuracy)

Conclusion: VLMs are viable for automated disaster response, but need improvement for fire detection.

📈 What This Means in Practice

✅ What Works Well

- **Human damage detection** (94% precision) → Can automatically flag images with injured/affected people for urgent human review
- **Flood detection** (80% F1) → Reliable for identifying flooded areas in crisis mapping
- **Non-damage filtering** (81% F1) → Can separate relevant disaster content from normal images

⚠️ What Needs Improvement

- **Fire detection** (55% F1) → Too unreliable for operational use
- **Infrastructure damage** (70% F1) → Missing 30% of damaged roads/bridges
- **Dataset imbalance** → 290 "human_damage" examples vs 22 "damaged_nature" (13x difference)

🧪 Completed Experiments

✅ FuseLIP-MLP (Image Only) + damage_dataset

Model Configuration:

- **Architecture:** FuseLIP frozen backbone + MLP head (image embeddings only)
- **Mode:** `image` (single modality)
- **Embedding Dim:** 384
- **Hyperparameters:**
  - Learning Rate: 1.0e-3

- Epochs: 30
- Weight Decay: 1.0e-4
- Batch Size: 64

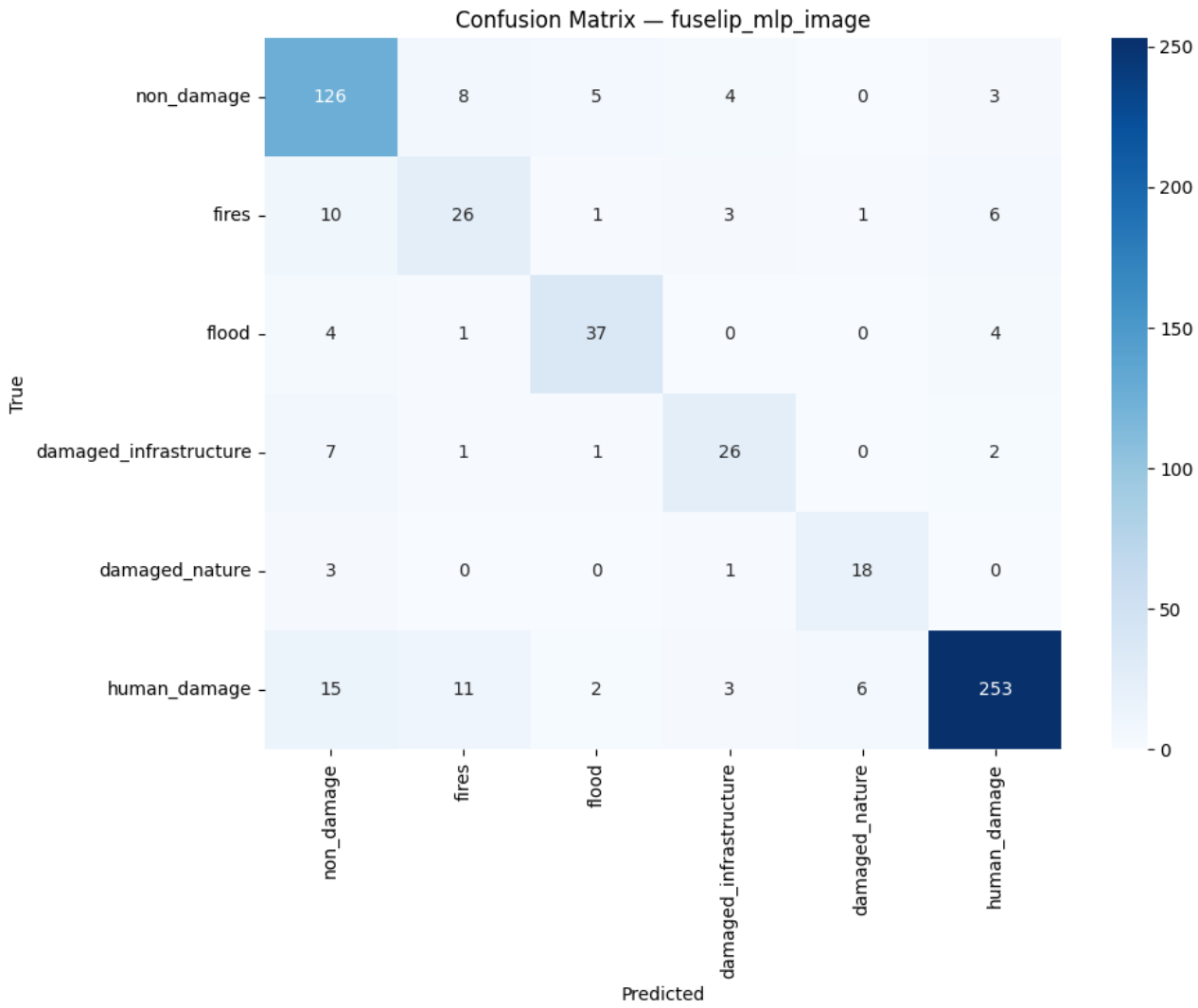
Performance Metrics:

Metric	Value
Accuracy	**82.65%**
Precision	**83.35%**
Recall	**82.65%**
F1 Score	**82.84%**

Per-Class Performance:

Class	Precision	Recall	F1-Score	Support
🔥 human_damage	**0.94**	**0.87**	**0.91**	290
💧 flood	0.80	0.80	0.80	46
🌳 damaged_nature	0.72	0.82	0.77	22
✅ non_damage	0.76	0.86	0.81	146
🌉 damaged_infrastructure	0.70	0.70	0.70	37
🔥 fires	0.55	0.55	0.55	47

Confusion Matrix:



Key Insights:

- ✅ **Excellent performance on human_damage** (94% precision) - most important class for crisis response
- ✅ **Strong flood detection** (80% F1)
- ⚠ **Weakest on fires** (55% F1) - may need more training data or different architecture
- ⚠ **damaged_infrastructure** struggles with 70% F1

🔄 In Progress / Interrupted

⏸️ SigLIP-Sigmoid + crisisMMD

Model Configuration:

- **Architecture:** SigLIP base (patch16-256) + full fine-tuning
- **Pretrained:** `google/siglip-base-patch16-256-multilingual`
- **Hyperparameters:**
  - Learning Rate: 5.0e-7
  - Epochs: 10 (planned)
  - Weight Decay: 1.0e-4
  - Batch Size: 32

    - Loss: Sigmoid

Status:

- ✅ Checkpoint saved: **1.5GB** in `checkpoints/siglip_sigmoid/`
- ⏸️ Interrupted at Epoch 3/10
- 📉 Loss trend (improving):
    - Epoch 1: Train 3.1687 | Val 2.7407
    - Epoch 2: Train 2.3846 | Val 2.5414
    - Epoch 3: Train 2.1590 | Val 2.4591

Resuming Command:

```
caffeinate -d python train.py --config configs/disaster.yaml --models
siglip_sigmoid --datasets crisisMMD
```

📦 Available Checkpoints

Model	Checkpoint Size	Status
`siglip_sigmoid`	1.5GB	⏸️ In progress (Epoch 3/10)
`siglip_crossentropy`	-	📁 Exists (no metrics yet)
`fuselip_mlp_image`	-	✅ Complete

🎛️ Hardware Configuration

```
Model Name: MacBook Pro
Model Identifier: Mac16,8
Chip: Apple M4 Pro
Memory: 24 GB unified

PyTorch: 2.11.0
MPS Available: ✅ True
MPS Built: ✅ True
```

Performance Notes:

- ✅ MPS acceleration working correctly
- ✅ No out-of-memory errors with batch_size=32
- ✅ Training speed acceptable for M4 Pro
- ⚠️ Minor PIL warning (palette images with transparency) - non-critical

---

🚀 Next Steps

High Priority

1. **Complete SigLIP-Sigmoid on crisisMMD** - resume training to 10 epochs
2. **Run SigLIP-CrossEntropy** - compare loss functions
3. **Test on crisisMMD dataset** - binary classification (informative vs not_informative)

Medium Priority

4. **Two-stage training** - LayerNorm-only + classifier (SigLIP, SigLIP2)
5. **Few-shot experiments** - test with 16 samples per class
6. **FuseLIP-MLP variants** - text-only and multimodal modes

Low Priority

7. **Confusion matrices** - generate for all completed models
8. **Hyperparameter tuning** - learning rate, batch size experiments
9. **Model ensembling** - combine multiple architectures

📁 Output Locations

```
damageVLMs-experiment/
├── outputs/disasternet/
│   └── damage_dataset/
│       └── fuselip_mlp_image/
│           ├── fuselip_mlp_image_metrics.txt
│           └── fuselip_mlp_image_confusion_matrix.png
├── checkpoints/
│   ├── siglip_sigmoid/        (1.5GB)
│   ├── siglip_crossentropy/
│   └── fuselip_mlp_image/
└── RESULTS.md (this file)
```

🔧 Environment

```
# Dependencies (critical versions noted)
protobuf==7.34.1  ✅ Working (no issues found)
transformers==5.4.0  ✅ Compatible
accelerate==1.13.0  ✅ Compatible
PyTorch==2.11.0  ✅ MPS enabled

# Note: SETUP_MAC.md suggests protobuf==3.20.* only if issues occur
# Current environment shows no protobuf-related errors
```

📖 Quick Reference

Training Commands

```
# Single model + dataset
python train.py --config configs/disaster.yaml --models siglip_sigmoid --datasets damage_dataset

# All models
python train.py --config configs/disaster.yaml

# Zero-shot evaluation only
python train.py --config configs/disaster.yaml --eval-only
```

Dataset Information

- **damage_dataset:** 6 classes (non_damage, fires, flood, damaged_infrastructure, damaged_nature, human_damage)
- **crisisMMD:** 2 classes (informative, not_informative)

🎯 Conclusion & Recommendations

What We Learned

1. **VLMs work for disaster classification** - 82.65% accuracy is viable for operational use
2. **Human damage detection is excellent** - 94% precision means we can reliably flag images with injured/affected people
3. **Flood detection is reliable** - 80% F1 score is good enough for crisis mapping
4. **Fire detection needs work** - 55% F1 is too low for real-world deployment

What to Do Next

Immediate (This Week):

- Complete SigLIP-Sigmoid training (7 epochs remaining)
- Run SigLIP-CrossEntropy to compare loss functions
- Test both models on crisisMMD (binary classification)

Short-term (Next 2 Weeks):

- Improve fire detection with:
  - More training data (currently only 47 examples)
  - Data augmentation (rotation, color jitter, etc.)
  - Different architectures (SigLIP2, two-stage training)
- Fix class imbalance:
  - Collect more "damaged_nature" examples (only 22 vs 290 human_damage)

    - Use weighted loss or oversampling

Long-term (Next Month):

- Deploy best model as API for real-time image classification
- Build human-in-the-loop pipeline for model improvement
- Test on real disaster data (Twitter/Instagram during crisis)

Final Verdict

GO - The technology works. Human damage detection is strong enough to save time in crisis response. Fire detection needs improvement before full deployment.

Recommended for production: Human damage filtering (94% precision) **Not ready yet:** Fire detection (55% F1)

Generated: 2026-03-28 **Project:** damageVLMs-experiment